You can imagine the tree structure like some kind of bolt and strings dangling structure, and then the whole operation can be described as grabbing the tree by node X and let all the other nodes dangle from it, with the caveat that we need to also move X's right child to node Y.

Before closing the discussion on rotations, it's important to remark that rotations always preserve BST constraints, but they do **not** preserve heap's invariants: rotations, in fact, can be used to fix broken treaps, but if applied to a valid tree, they will break the priority constraints on the node to which they are applied.

3.3.2 A Few Design Questions

Treaps are heaps, which in turn are special trees with a dual array representation: as we have seen in chapter 2, we can implement a heap using an array, a more space-efficient representation that also exploits locality of reference.

Can we also implement a treap using an array? I encourage you to take a minute and think about this question, before moving on and reading the answer: what would be the PROs and CONs of using an array versus a tree, what could be the pain points using a tree?

The issue with the array representation is that it's not particularly flexible: it works well if we only swap random elements and remove/add only from the array's tail; if, instead, we need to move elements around, it's a disaster! For instance, even inserting a new element in the middle of the array causes all the elements after it to be moved, for an average O(n) swaps (see figure 3.7).

The key point with heaps is that they are complete, balanced and left-aligned trees, which is possible because heaps don't keep a